2MA100 - Devoir Maison 1

Les exercices de ce devoir maison noté sur 20 points doivent être rendus au choix au format **Notebook** (`.ipynb`) ou **Script** (`.py`) en un ou plusieurs fichiers au plus tard le **12 mars 2021** à 20h00 sur moodle.

Attention Plagiat.

Les devoirs maisons doivent être codés et rendus de manière individuelle. A titre d'information, voici une définition du plagiat adaptée du Memento de l'Université de Genève :

Le plagiat consiste à insérer, dans un travail académique, des formulations, des phrases, des passages, des morceaux de code, des images, de même que des idées ou analyses repris de travaux d'autres auteurs, en les faisant passer pour siens.

En particulier, le copier-coller à partir de sources trouvées sur Internet ou sur des travaux d'autres étudiant·es sans citer les sources est considéré comme du plagiat et implique une note zéro. Le plagiat constitue également une tentative de tricherie sanctionnée par le règlement de l'université. La solution est d'indiquer dans vos devoirs tout de ce qui ne vient pas de vous en mentionnant les sources (page Internet, livres, autre étudiant·e,...). Tous les fichiers rendus seront analysés automatiquement avec un logiciel de détection des similarités (entre étudiant·es et depuis Internet).

Les modules Numpy et Matplotlib peuvent être utiles dans certains exercices :

```
import numpy as np
import matplotlib.pyplot as plt
```

Exercice 1 : Formule de Leibniz

On rappelle la formule de Leibniz :

$$\frac{\pi}{4} = \sum_{k=0}^{\infty} \frac{(-1)^k}{2k+1},$$

et le but est de l'utiliser pour obtenir une approximation de π. Pour un entier $n \geq 0$, on note

$$S_n = \sum_{k=0}^{n} \frac{(-1)^k}{2k+1}.$$

a) Écrire une fonction qui, à n donné, calcule S_n.

Solution : Il suffit de sommer les termes jusqu'à n, sans oublier le dernier terme :

```
def Sn(n):
    S = 0
    for k in range(n+1):
        S += (-1)**k / (2*k+1)
    return S
```

b) Écrire une fonction dépendant de ε qui renvoie le plus petit entier n tel que $|\pi - 4S_n| < \varepsilon$.

Solution : Il suffit de sommer tant que la condition n'est pas satisfaite :

```
def approximation(eps):
    # initialisation
    n = 0
    S = 1
    # boucle tant que la condition pas satisfaite
    while  abs(np.pi-4*S)>eps:
        n += 1
        S += (-1)**n / (2*n+1)
    return n
```

c) Tracer l'évolution de $|\pi - 4S_n|$ en fonction de n pour $3 \leq n \leq 100$.

Solution : Une façon de faire est d'utiliser la fonction `Sn` en pernant soin de la vectorialiser pour l'appliquer sur un vecteur :

```
n = range(3, 101)
erreur = np.abs(np.pi - 4*np.vectorize(Sn)(n))
plt.title("Erreur $|\pi - 4 S_n |$ en fonction de $n$")
plt.xlabel('$n$')
plt.plot(n, erreur)
```

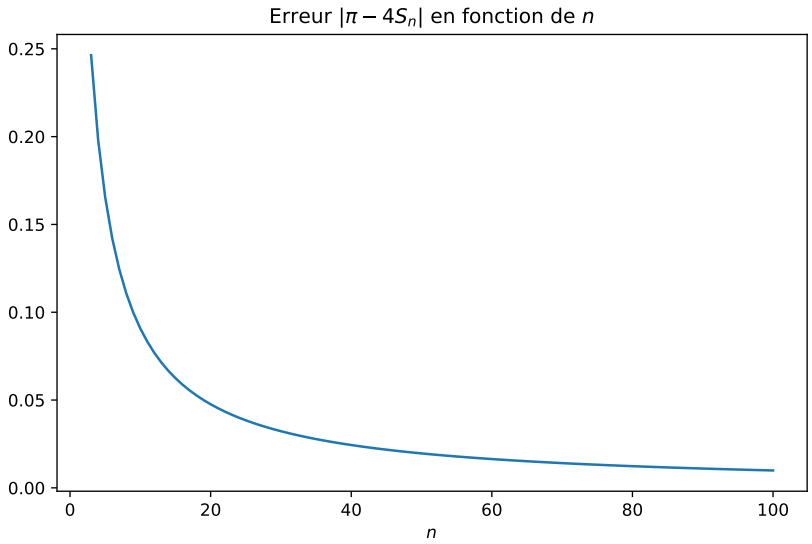
d) Vos implémentations des deux questions précédentes sont-elles optimales quand au nombre d'opérations effectuées ?

Solution : L'implémentation de la question **b)** est optimale dans le sens ou la somme S_n n'est pas calculée à chaque fois. Une implémentation non-optimale serait par exemple :

```
def approximation2(eps):
    # initialisation
    n = 0
    S = 1
    # boucle tant que la condition pas satisfaite
    while  abs(np.pi-4*S)>eps:
        n += 1
        S = Sn(n)
    return n
```

Par contre l'implémentation de **c)** n'est pas optimale, en effet la somme est recalculée pour chaque valeur de n. Une façon optimale quand au nombres d'opérations serait par exemple de construire récursivement le vecteur des sommes partielles :

```
nmin = 3
nmax = 100
S = np.zeros(nmax+1)
S[0] = 1
for n in range(1,nmax+1):
    S[n] = S[n-1] + (-1)**n / (2*n+1)
erreur = np.abs(np.pi - 4*S[nmin:nmax+1])
plt.title("Erreur $|\pi - 4 S_n |$ en fonction de $n$")
plt.xlabel('$n$')
plt.plot(range(nmin,nmax+1), erreur)
```



Exercice 2 : Séquençage de l'ADN

Le séquençage de l'ADN (une chaîne de caractères) consiste à déterminer l'ordre d'enchaînement des nucléotides pour un fragment d'ADN donné.

a) Etant donné un séquençage d'ADN (par exemple, `S = "ACCTG"`), écrire une fonction retournant un dictionnaire comptant le nombre d'occurrences de `A`, de `C`, de `G` et de `T`.

Solution : La méthode `get` d'un dictionnaire permet d'ajouter un second paramètre pour retourner cette valeur lorsque la clef n'existe pas encore :

```
def occurrences(S):
    dic={}
    for e in S:
        dic[e]=dic.get(e,0)+1
    return dic
```

Pour tester :

```
occurrences("ACTGGGTA")
```

b) Lorsqu'une bactérie est atteinte par un virus, la réplication de l'ADN peut être affectée de manière aléatoire. Chaque nucléotide (`A`, `C`, `G` ou `T`) va selon une certaine probabilité être recopiée de la manière suivante : Chaque `A` peut être répliqué comme 3 `A`, chaque `C` peut être répliqué comme 4 `C`, chaque `G` peut être répliqué comme 4 `G`, et chaque `T` peut être répliqué comme 3 `T`.

Écrire une fonction `infect(S,D)` qui étant donné une séquence `S` d'ADN et un dictionnaire `D` donnant la probabilité de réplication de chaque nucléotides produira aléatoirement une séquence infectée. Par exemple, si `S = "ACCTG"` et `D = {'A':0.2,'C': 0.5,'G': 0.3,'T': 0.1}`, la première lettre (un `A`) a 20% de chance d'être répliquée comme 3 `A`, la seconde (un `C`), 50% de chances de devenir 4 `C`, etc.

Solution :

```python
import random

def infect(S,D):
    out=""
    for e in S:
        a=random.random()
        proba=D.get(e,0)
        if a<=proba:
            if e=="A":
                out=out+"AAA"
            elif e=="C":
                out=out+"CCCC"
            elif e=="G":
                out=out+"CCCC"
            elif e=="T":
                out=out+"TTT"
        else:
            out=out+e

    return out
```

Pour tester :

```python
S="CCAGT"
D={'A':0.2,'C':0.5,'G':0.3,'T':0.1}
infect(S,D)
```

c) Dans la suite on appelle `A` et `B` deux séquences d'ADN de respectivement m et n nucléotides que l'on veut comparer.

Pour déterminer l'alignement de score maximal de `A` et `B`, et permettre de trouver la meilleure ressemblance parmi un ensemble de comparaisons, on peut utiliser l'algorithme de Needleman-Wunsch. Une matrice F des similarités est utilisée et ses éléments aux positions (i, j) sont notés $F_{i,j}$ pour $0 \leq i \leq m$ et $0 \leq j \leq n$. Il y a une colonne pour chaque caractère de la séquence `B`, et une ligne pour chaque caractère de la séquence `A`.

Au fur et à mesure de la progression de l'algorithme, $F_{i,j}$ se verra affecter le score optimal pour l'alignement des i premiers caractères de A avec les j premiers caractères de B. L'algorithme se présente ainsi :

- Pour l'initalisation, une matrice F avec $m+1$ lignes et $n+1$ colonnes indexée à partir de zéro est créée pour comparer des séquences `A` et `B` de respectivement m et n nucléotides. On initialise la première ligne et la première colonne avec des zéros.
- Pour chaque $i = \{1, ..., m\}$ et chaque $j = \{1, ..., n\}$, les valeurs de F sont définies récursivement par

$$F_{i,j} = \max\big(F_{i-1,j-1} + S_{i-1,j-1}, F_{i-1,j} - 1, F_{i,j-1} - 1\big) \quad \text{où} \quad S_{i,j} = \begin{cases} 1 & \text{si } A_i = B_j \\ 0 & \text{sinon} \end{cases}.$$

- Pour finir, le score maximal d'alignement est le maximum des coefficients de la matrice F, $d = \max_{i,j}(F_{ij})$.

Écrire une fonction `needleman_wunsch(A,B)` qui étant donné deux séquences d'ADN renvoie le score maximal d'alignement entre ces deux chaînes. Tester avec les séquences `A = "GAATTCAGTTA"` et `B = "GGATCGA"` pour lesquelles le score maximal d'alignement est 4.

Solution : Il est relativement pratique d'utiliser Numpy :

```python
def nw(A,B):
    m=len(A)+1
    n=len(B)+1
```

```
    F = np.zeros((m+1,n+1))
    for i in range(1,m):
        for j in range(1,n):
            S = 1 if A[i-1]==B[j-1] else 0
            F[i,j] = max(F[i-1,j-1]+S, F[i-1,j]-1, F[i,j-1]-1)
    return np.max(F)
```

On retrouve bien le résultat annoncé :

```
nw("GAATTCAGTTA","GGATCGA")
```

d) Ouvrir le fichier `adn.txt` en lecture. Soit `S = "CGTCCGGTTTCGATTGGAGGGCTCGGAA"` une séquence d'ADN suspectée responsable d'une mutation. Récupérez les 8 personnes dans le fichier ayant la plus forte similarité avec cette séquence, et afficher leurs taux de similarités, leurs noms et prénoms.

Indication : Pour lire le fichier `adn.txt` utiliser la fonction `open` dont la documentation est disponible ici. Par exemple pour afficher toutes les lignes du fichier commencant par la lettre "A" :

```
with open("adn.txt","r") as file:
    for ligne in file:
        if ligne[0]=="A": print(ligne)
```

Solution :

```
# ADN du suspect
S="CGTCCGGTTTCGATTGGAGGGCTCGGAA"

# Lecture du fichier
with open("adn.txt","r") as file:
    d=[]
    for ligne in file:
        L = ligne.split()
        nom=L[0]
        prenom=L[1]
        ADN=L[2]
        taux=nw(ADN,S)
        d.append([taux,prenom,nom])

# Donne les 8 premiers suspects
d.sort(key=lambda x: -x[0])
d[0:8]
```

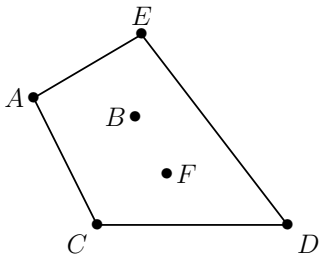
e) [bonus] Pour deux séquences de tailles n et m, quelle est la complexité de l'algorithme de Needleman-Wunsch ?

Solution : La complexité est en $O(mn)$.

Exercice 3 : Calcul d'enveloppe convexe

Le but de l'exercice est d'implémenter un algorithme qui permet de calculer l'enveloppe convexe d'un ensemble fini $\mathcal{E}$ de points de $\mathbb{R}^2$.

On pourra représenter un point de $\mathbb{R}^2$ par un 2-`tuple` de flottants, un `complex`, ou toute structure qui paraîtra adaptée à l'exercice. Un ensemble de points pourra être représenté par un `set` ou une liste de points, selon la structure choisie pour les points. L'enveloppe convexe d'un ensemble de points pourra être représentée par une liste de points successifs qui permettent de la tracer. La figure suivante représente l'enveloppe convexe $[A, E, D, C]$ des points $\{A, B, C, D, E, F\}$:



Le but est d'implémenter l'algorithme de la marche de Jarvis. L'idée est de construire la liste représentant l'enveloppe convexe successivement, en partant d'un point d'abscisse minimale (le point A dans la figure).

a) Écrire une fonction `premier_point(E)` qui étant donné un ensemble de points `E` renvoie un point d'abscisse minimale.

Solution : On choisit de représenter les points en plan comme des nombres complexes :

```
def premier_point(E):
    # indice du point ayant la plus petite partie réelle
    i = np.argmin(np.real(E))
    # on renvoie le point correspondant
    return E[i]
```

Les points suivants sont construits en "tournant" autour de l'ensemble $\mathcal{E}$ toujours dans le même sens. Plus précisément, si X est un point de l'enveloppe convexe et que $Y_1, Y_2 \in \mathcal{E}$, on dit que Y_1 est **à gauche** de Y_2 pour le point X si et seulement si l'angle orienté $\widehat{Y_2XY_1}$ est dans $[0, \pi]$. Par exemple sur la figure, en considérant $X = A$, on a : B est à gauche de F, qui est à gauche de C. Pour A, le point *le plus à gauche* est E, qui est le second point de la liste qui donne l'enveloppe convexe. En réitérant jusqu'à retomber sur A, on trouve l'enveloppe convexe en entier.

On pourra supposer pour faire simple que trois points de l'ensemble $\mathcal{E}$ ne sont jamais alignés.

b) Écrire une fonction `gauche(X, Y1, Y2)` qui renvoie `True` si `Y1` est à gauche de `Y2` pour le point `X` (et renvoie `False` sinon). On ferra en sorte que `gauche(X, Y, X)` renvoie `True` et que `gauche(X, X, Y)` renvoie `False`.

Solution :

```
def gauche(X, Y1, Y2):
    if X==Y2:
        return True
    if X==Y1:
        return False
    else:
        # la condition sur l'angle se ramène à la formule suivante
        return np.angle((Y1 - X) / (Y2 - X)) >= 0
```

c) Écrire une fonction `prochain_point(E, X)` qui, étant donné un ensemble de points `E` et un point `X` renvoie le point `Y` le plus à gauche pour `X`.

Solution :

```
def prochain_point(E, X):
    Y = X # tout point différent de X sera plus à la gauche de X que X lui-même
    for Z in E:
        # si on trouve un point plus à gauche que Y, on le remplace
```

```
        if gauche(X, Z, Y):
            Y = Z
    return Y
```

d) Écrire une fonction `enveloppe(E)` qui renvoie l'enveloppe convexe de l'ensemble `E`.
Solution :

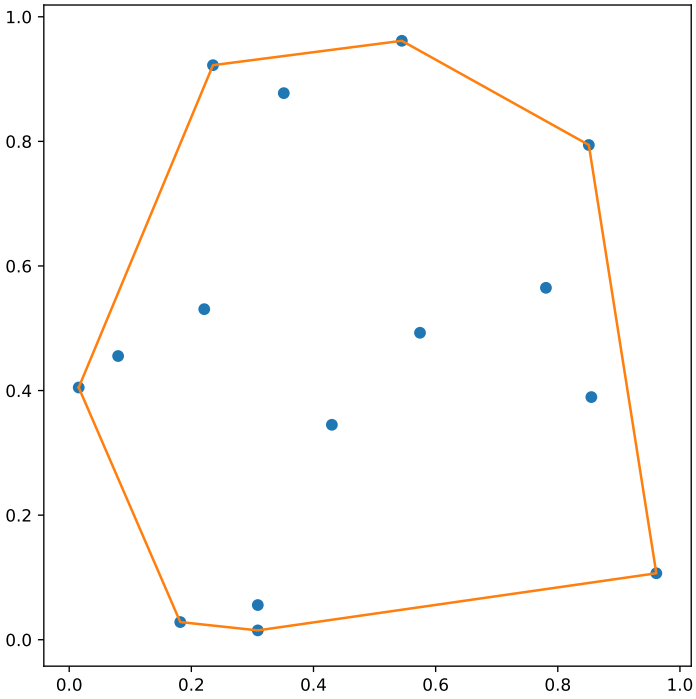
```
def enveloppe(E):
    # on initialise l'enveloppe avec le premier point
    A = premier_point(E)
    env = [A]
    while True:
        # on calcule le prochain point de l'enveloppe, à partir du dernier trouvé
        B = prochain_point(E, env[-1])
        # si c'est le point de départ, on a fini
        if B == A:
            return env
        # sinon, on le rajoute à l'enveloppe et on continue
        env.append(B)
```

e) En utilisant par exemple la fonction `scatter` de Matplotlib pour écrire une fonction permettant de représenter un ensemble de points et tracer son enveloppe convexe. Tester avec un ensemble de 15 points aléatoires dans $[0, 1]^2$.
Solution :

```
def plot_enveloppe_convexe(E):
    # on récupère la liste des points de l'enveloppe convexe
    env = enveloppe(E)
    # on fait boucler
    env.append(env[0])
    # puis on transform ça en array numpy
    env = np.array(env)
    # affichage des points de E
    plt.scatter(np.real(E), np.imag(E))
    # affichage de l'enveloppe convexe
    plt.plot(np.real(env), np.imag(env), color='C1')
    return
```

Pour tester, on génère 15 points aléatoirement dans le plan et on en dessine l'enveloppe convexe :

```
n = 15
E = np.random.random(n) + 1j * np.random.random(n)
plt.figure(figsize=(7,7))
plot_enveloppe_convexe(E)
```


1.0
0.8
0.6
0.4
0.2
0.0
0.0
0.2
0.4
0.6
0.8
1.0